On the Use of Evolutionary Algorithms for Test Case Prioritization in Regression Testing Considering Requirements Dependencies

Andreea Vescan
andreea.vescan@ubbcluj.ro
Babes-Bolyai University
Faculty of Mathematics and Computer Science
Romania

Camelia Chisalita-Cretu
maria.chisalita@ubbcluj.ro
Babes-Bolyai University
Faculty of Mathematics and Computer Science
Romania

Camelia Serban
camelia.serban@ubbcluj.ro
Babes-Bolyai University
Faculty of Mathematics and Computer Science
Romania

Laura Diosan
laura.diosan@ubbcluj.ro
Babes-Bolyai University
Faculty of Mathematics and Computer Science
Romania

ABSTRACT

Nowadays, software systems encounter repeated modifications in order to satisfy any requirement regarding a business change. To assure that these changes do not affect systems' proper functioning, those parts affected by the changes need to be retested, minimizing the negative impact of performed modifications on another part of the software. In this research, we investigate how different optimization techniques (with various criteria) could improve the effectiveness of the testing activity, in particular the effectiveness of test case prioritization.

The most efficient test schedules are identified by using either a Greedy algorithm or a Genetic Algorithm, optimizing a quality function that incorporates single or multiple criteria. Both functional requirements (with the existing dependencies between them) and non-functional requirements (i.e. quality attributes for test cases) are integrated with the quality assessment of a test order.

Therefore, the conducted experiments considered various criteria combinations (faults, costs, and number of test cases), being applied to both theoretical case studies and a real-world benchmark. The conclusion of the experiments shows that the Genetic Algorithm outperforms the Greedy on all considered criteria.

CCS CONCEPTS

• **Software and its engineering → Software defect analysis**; • **Computing methodologies → Artificial intelligence**.

KEYWORDS

Regression testing, Test case prioritization, Requirements dependencies, Metrics, Quality, Optimization.

ACM Reference Format:
Andreea Vescan, Camelia Chisalita-Cretu, Camelia Serban, and Laura Diosan. 2021. On the Use of Evolutionary Algorithms for Test Case Prioritization in Regression Testing Considering Requirements Dependencies. In *Proceedings of the 1st ACM International Workshop on AI and Software Testing/Analysis (AISTA '21), July 12, 2021, Virtual, Denmark.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3464968.3468407

AISTA '21, July 12, 2021, Virtual, Denmark

ACM ISBN 978-1-4503-8541-1/21/07…$15.00
https://doi.org/10.1145/3464968.3468407

1 INTRODUCTION

Regression Testing (RT) is the activity that aims to detect and to minimize the negative impact of performed changes to other parts of the software. More precisely, after a change is applied, the already working program is retested (in some selected areas) to detect whether the actual change achieved its intended purpose, without adding new bugs.

The state of the art literature [21], [13] on RT highlights several strategies like Test Case Prioritization (TCP) [24], Test Case Selection (TCS) [35], and Test Case Minimization (TCM) [15], [36], [37]. This paper addresses TCP strategy, however our future research on RT aims to design a broader framework that combines the above mentioned RT strategies. The use of different strategies in combination will allow to benefit from each other's strengths.

Most existing approaches on RT that use TCP consider only the test requirements. Following the reasoning above, that a change in one part of the system may have unexpected effects in other parts, i.e., due to dependencies between requirements, we reflected and concluded that when prioritizing the test cases due to a change in the requirement (reflected in the source code) those requirements dependencies should play an important role in the chosen strategy. To the best of our knowledge, a few researches like Srikanth et al. [30] and Salehie et al. [28] considered functional requirements when selecting test cases for RT, and a few considered dependencies between requirements [14], [23]. Also, our TCP approach is structured in three steps: data synthesis, filtering and optimization. The outcome of each phase is presented in this paper, with emphasis on the benefits of the filtering step. Having available the requirement dependencies and link between the test cases and requirements, and knowing the performed change on the requirements (and not only in the source code), one may be able to prioritize the test cases that are effected by the change and also consider the other requirements that depend on the same changed requirements, thus reducing the time execution and in the same time increasing the change to find more bugs.

The paper by Vescan et al. [33] proposes the TCP formalization that considers not only test requirements but also functional requirements and dependencies between the functional requirements.

Our approach builds on this one, considering new metrics in the analysis, validating the proposals using two additional case studies (one more synthetic example and the *grep* project) [1], and we also compared more optimisation methods (Greedy and Genetic Algorithms). Thus, the first novelty of our approach considers the dependencies between functional requirements as part of the test case prioritization, and the second novelty concerns the minimization of the test cases to be considered in order to achieve high covered faults.

The paper is organized as follows: Section 2 provides a formalization of the TCP Problem and a literature review regarding similar approaches. Section 3 describes our methodology for the TCP Problem in the RT context. Section 3.1 covers the identification of the input data required for our analysis on TCP and the definition of the metrics used in our approach. Section 3.2 addresses the filtering strategies of those inputs that are related to the modified requirements, while Section 3.3 shortly describes the optimization methods and the criteria used for prioritization. Section 4 describes in more detail the evolutionary algorithms. Section 5 presents the experimental analysis, the data for each case study, the applied methodology, and the obtained results. Section 6 discuses threats to validity that can affect the results of our study. Conclusions and future directions are emphasized in Section 7.

2 BACKGROUND. LITERATURE REVIEW. FORMALIZATION

This section frames the context and the formalization of our investigated problem, and also a literature review related to our proposed approach.

2.1 Regression Testing and Test Case Prioritization

Regression testing is a type of testing that aims to increase the confidence that changes applied on a software system do not affect the already working functionalities [13]. In order to find the faults early when RT is performed, the developed approaches aim to reduce the cost and time amount required to run the test cases. These objectives were attained through the following techniques [37]: test suite minimization (TSM), test case selection (TCS), and test case prioritization (TCP).

This paper addresses the TCP technique listed above, defining it as the core of our current research inquiry on RT. Regression testing based on the TCP techniques re-orders the test cases in such a way that high priority test cases can be executed first. The priority of a test case can be determined based on the coverage information from structural code, fault detection ability of test cases, test cases' execution cost, or other explicitly defined testing goals.

It is also obvious that coverage and fault-based priority have a relationship with each other. On the other hand, the execution cost of test cases has some weak dependency on the coverage of the test cases in test suites. Regarding RT, there is always the concept of trade-off, which creates the two side dilemma. *What we should do?* vs *What we can afford to do?*, which is a very common quote that applies to RT in a very specific way.

It was observed and reported that time constraints played a significant role in the aforementioned trade-offs, which refers to the cost criterion effectiveness within the TCP techniques and the relative cost-benefit compromise between such techniques. The results suggest that manipulating the cost criterion within the TCP techniques can result in an improved RT. If not all the regression tests can be executed, then the cost analysis can be lowered by simply performing partial prioritization, which means that essentially we prioritize fewer test cases [7]. We mention that by cost we refer to the cost execution of a test case and not the cost of the optimization techniques.

2.2 Test Case Prioritization - Literature Review

This section briefly surveys the existing TCP algorithms and related work in this area. We have selected the papers that approached the same problem as we did. The paper-based analysis considered various aspects referring to: the used algorithm/method, objectives used in the prioritization, complexity of the employed case studies and comparison method with other approaches. We have followed the steps from [19] when selecting the papers for related work.

Most of the *algorithms/methods* used for TCP are Greedy, Hill Climbing and GA. Greedy Algorithms have been widely employed for TCP [21], incrementally adding test cases to an initially empty sequence. The choice on which test case to add next affects the maximum/minimum value for the desired metric (e.g., cost of a test case metric). Rothermel et al. [27] point out that this greedy prioritization algorithm may not always choose the optimal test case ordering for other criteria.

The application of Genetic Algorithms has been proved to be effective [28], [29], [6] for the TCP problem in the context of RT.

As *objectives*, some approaches optimise the cost of a test suite and the number of faults discovered earlier [16], while other approaches considered the APFD metric [16],[29], [17] or the code coverage [16], [35], [21]. Several approaches consider the TCP as a single objective optimisation problem [29], [17], while [35], [11], [30] deal by the multi-objective TCP's perspective.

Referring to the *case study* complexity and comprehensiveness, most of the approaches considered in their empirical studies only code coverage, cost, faults, or APFD metrics. One approach considered a non-functional attribute, i.e. "customer priority" [30] and only one considered requirements [28].

2.3 Test Case Prioritization Problem

2.3.1 Problem Formal Statement . TCP's goal is to find an optimal order in which to execute test cases. The ideal ordering of test cases would be the one that maximizes the rate of fault detection. However, since the fault information is not known to the tester in advance, prioritization techniques have to depend on surrogates.

Various objective functions are available in the literature: ***statement coverage*** ([11, 21]) – a surrogate for fault detection capability; ***δ-coverage*** ([11, 35]) – difference of the statement coverage between two consecutive versions; ***APFD*** ([9, 25, 26]) – which rewards orderings with earlier fault detection abilities; ***fault history coverage*** ([11, 16, 17, 29]) – if we have one past fault, we can say that a test covers that fault if it is able to detect it. Fault history coverage has been used several times in test case minimization [35, 39], but has not be used in prioritization. ***execution cost*** ([28, 35]) – with two approaches: the number of steps performed or the wall-clock

time, which depends on the underlying hardware and operating system.

Our regression testing approach defines TCP problem as follows, emphasizing in bold fonts the supplementary elements relative to Rothermel definition available in [25]:

Provided: **a program** P **that implements a set of functional requirements** R, a test suite T, the set of permutations of T denoted by PT; **the program** P' **as a version of** P **that modifies a set of requirements** MR **included in** R**, a set of faults** F **seeded in** P', an objective function $f : PT \rightarrow \Re$.

Ensures: to find a permutation $\sigma \in PT$ such that $(\forall \sigma')(\sigma' \in PT)(\sigma' \neq \sigma)[f(\sigma) \geq f(\sigma')]$.

This objective function f may be the APFD metric [26] that needs to be maximized, an usability metric to be optimized, or any other goal set according to an established testing strategy.

The terms and notations of the investigated problem are formally described below, with emphasis on the existing relationships.

- P – the program or SUT;
- R – the set of functional requirements of program P, where $R = \{r_1, r_2, \ldots, r_n\}$;
- $RDR : R \times R \rightarrow \{0, 1\}$ – the dependencies between the requirements;
- T – the test suite, where $T = \{t_1, t_2, \ldots, t_m\}$ consists of tests;
- $TR : T \times R \rightarrow \{0, 1\}$ – the dependencies between test cases and requirements;
- F – a faults set, where $F = \{f_1, f_2, \ldots, f_q\}$ are the faults to be seeded in the program P;
- $TF : T \times F \rightarrow \{0, 1\}$ – the fault traceability mapping, where:

$$TF(t_j, f_k) = \begin{cases} 1, & \text{if test } t_j \text{ detects fault } f_k \\ 0, & \text{if test } t_j \text{ does not detect fault } f_k \end{cases}$$

$$\text{where } j \in \{1, \ldots, m\},\ k \in \{1, \ldots, q\}.$$

- $C : T \rightarrow \mathbb{N}^*$ – the test execution time expressed as a test cost for each $t_j \in T, i = \overline{1, m}$; this cost is considered to be unchanged on each test run;
- MR – the set of modified requirements of the program P, $MR \subseteq R$;
- $\sigma_T = (t_1, t_2, \ldots, t_m)$ – a permutation for the test suite T;
- PT – the set of all permutations; σ_i associated with the test suite T, $i = \overline{1, m}$
- P' – a changed version of program P that includes the seeded faults from the faults set F.

2.3.2 Objective Functions - Metrics Definitions. We aim to explore the test case ordering criteria regarding non-functional requirements. Therefore, to define metrics in order to quantify these aspects, focusing on the usability quality attribute of a test case. Usability is expressed in terms of effectiveness and efficiency. Thus, we define a series of optimizers as:

(1) *APFD with requirements* – meaning the smallest number of tests required to cover all requirements and all faults;
(2) *effectiveness* – to measure the completion rate;
(3) *efficiency* – to measure the time required to complete a task.

Various metrics [31], [11], [30], [6], [26] have been proposed so far to measure the effort implied in software testing activities and to assure that every part of the system works as expected. Most of these papers considered two important characteristics used as optimization criteria in RT: *Effectiveness* and *Efficiency*.

In spite of this great number of papers that quantify software testing activity, few of them [26] defined metrics for *Effectiveness* and *Efficiency*. Regarding the metrics used to quantify *Effectiveness* one of the most widely used metrics is **APFD** [26], which measures *the effectiveness of TCP as the rate of fault detection* achieved by the produced ordering of test cases. Metrics related to *Efficiency* are expressed in time needed to execute a test case.

Thus, there is still a need for metrics to address *Effectiveness* and *Efficiency*. These quality characteristics are related in ISO/IEC 9126-4 Metrics [2] with usability quality factor. Starting from this idea we adapted some metrics for quantifying *Effectiveness* and *Efficiency* and we defined several new ones. In what follows, we present the definition of our proposed metrics for usability adapted from ISO/IEC 9126-4 Metrics [2].

Our Adapted Metrics for Effectiveness and Efficiency

Metrics for quantifying usability, an important quality factor defined by ISO/IEC 9126-4 Metrics [2], are defined in terms of the number of tasks that are successfully completed by the user. We consider these metrics important in regard to testing activity, and in the context of RT we have adapted these definitions.

The proposed metrics for Effectiveness of a test case and of a test suite, respectively, are described in Equations 1 and 2, while Equations 3 and 4 express the efficiency of the testing process (relative to a test case and a test suite, respectively).

$$\text{EFFE} = \frac{\text{Number of faults discovered}}{\text{Total number of faults}} \times 100 \tag{1}$$

$$\text{Effe\_For\_All} = \frac{\text{No. of TCs detected at least 1 fault}}{\text{Total no. of TCs undertaken}} \times 100 \tag{2}$$

$$\text{EFFI} = \frac{\text{Time to execute the test case}}{\text{Number of faults}} \tag{3}$$

Remark. When the number of faults for a test case is 0, the value of metric is computed as the maximum times for all test cases.

$$\text{TC\_Effi\_For\_All} = \frac{\sum_{j=1}^{m} \frac{faultFound_j}{time_j}}{m} \tag{4}$$

where:

- $faultFound_i = \begin{cases} 1, \text{if the test case } t_j \\ \text{discovered at least one fault,} \\ 0, \text{otherwise;} \end{cases}$
- $time_j$ represents the time to execute the test case t_j.

2.4 Contributions on TCP Perspective

Considering the definitions of RT and TCP, the literature review and the proposed formalization, we resume the following perspectives and findings: in relation to these existing approaches, our proposal uses similar methods, but there are some differences mentioned below. As far as we know, there are few references in the literature [3] that considered the dependencies between requirements in test case prioritization algorithm, this being a strength of our proposal. Additionally, we adapted some new metrics for the usability quality attribute, i.e., effectiveness and efficiency of a test case. Another important difference is evaluating the rate of fault detection using the

APFD metric. Our approach considers this metric as objective function in the optimization process, whereas the related approaches use this metric to assess the obtained solutions.

In what follows, we present the methodology and the detailed steps of our approach, backed up by numerical results and validate it against a set of benchmarks. We also highlight connections and differences with other optimization approaches used in previous work on TCP.

3 RESEARCH METHODOLOGY

In our proposed method, the following steps are performed to obtain a permutation of test cases that improve the effectiveness of test cases prioritization problem: (1) Data synthesis; (2) Filtering based on modified requirements and Metrics Computation; (3) Prioritization - search-based approach.

To bring clarity to our approach, Figure 1 summarizes our approach's main steps, underlining their constituent elements and how they are connected. The following subsections describe each step in detail, highlighting several novelties throughout. We have used colors to group elements from the data synthesis step to the actions that define our approaches.

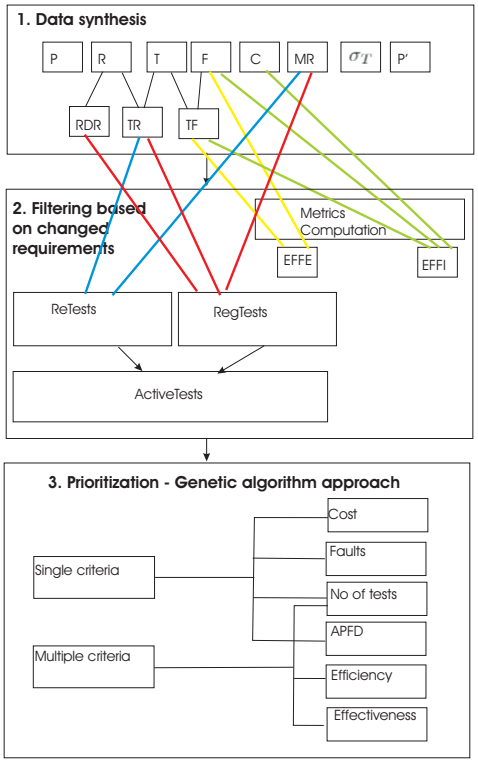

Figure 1: Overview of our research methodology

3.1 Data Synthesis

Data synthesis refers to the identification of the input data required for our analysis on TCP. We establish the dependencies between requirements (*RDR*), the associations between test cases and requirements (*TR*), the faults revealed by a specified test case (*TF*), and the cost of a test case (*C*).

The first novelty of our approach is given by considering dependencies between functional requirements. When prioritizing test cases, we take into account not only the test cases that are linked to modified requirements, but also we identify test cases related with those requirements that depend on the modified ones.

3.2 Filtering Based on Modified Requirements and Metrics Computation

Filtering the data set refers to processing and taking into account only those values that are related to the modified requirements. Here, we establish the set of test cases to be run. These test cases are obtained by bringing together `ReTest` set and `RegTest` set.

The `ReTest` set consists of the test cases that were mandatory after performing modifications, i.e. the tests that exercise the requirements that were modified. In Figure 1 the *ReTests* is linked to both sets of modified requirements *MR* and *TR* matrix that relates the tests to the requirements. The `RegTest` set considers the test cases that are related with the requirements that depend on those that were modified. In Figure 1, the *RegTests* is linked to three basic elements, i.e. the set of modified requirements *MR*, the *TR* that relates tests to requirements and the *RDR* that describes the dependencies between requirements. Therefore, $ActiveTests = ReTest \cup ReqTest$. This allowed us to provide **the second novelty** by establishing a minimum number of test cases to be run such that all faults involved by the tested requirements are revealed.

The defined *EFFE* and *EFFI* metrics are computed in this step because we consider the total number of faults discovered by the *ActiveTests* set and not the total number of faults existing in P.

3.3 Prioritization Using a Search-Based Approach

Our work on TCP is based on a search algorithm, in fact an evolutionary one, considering both single and multi-criteria optimisation objectives. Therefore, a solution for TCP has to provide a specific test case ordering out from `ActiveTests` by taking into account one or more of the following optimisation criteria:

(1) *testing cost* – meaning the effort required to run the corresponding tests, with the intent to minimize it;
(2) *number of tests* – indicating the minimal number of tests that cover all the faults;
(3) *a weighted combination of more criteria* - three objectives (of equivalent importance) are considered: the number of selected tests, the total effectiveness and total time efficiency.

The research questions that our approach investigates are stated in the following:

RQ1. Optimisation Quality: Does multi-objectives approach produce better quality solutions when compared to the single objectives algorithm? If so, how much better?

RQ2. Testing Effectiveness: How effective are the prioritized test suites in terms of early fault detection or other important measures?

To answer those questions we first present the details of our approach, i.e. using GA to solve the TCP, emphasizing the representation and genetic operators.

4 PROPOSED GA-BASED APPROACH FOR TCP

We have until now specified the TCP problem and we further present the algorithm for solving it. The prioritization problem could be considered an optimization one; a single criterion or more criteria (number of tests, execution time, various types of coverage, different efficiency metrics) could be taken into account.

The multi-objective optimisation methods can be classified by taking into account two aspects: searching (the optimisation of the objective function) and making decision (the decision process of selecting the best solution by a trade-off of various criteria): *a priori* methods, when the optimisation preferences must be declared before the searching process starts, and *a posteriori* methods, when the first step is the searching one and the second step is the decision making.

The last of the *a posteriori* methods' stage, that of choosing a single solution from a large set of possible good solutions (from the Pareto front), could be hard. Furthermore, the generation of the Pareto front can be difficult and computationally expensive. Therefore, we decided to involve in our approach an *a priori* multi-objective optimisation method. Of the two classes of *a priori* methods we decided to use the first one; in fact, the weighted global criterion method in which all objective functions are combined to form a single function, by using a weighted-sum model whose parameters expose the relative importance of a given objective ($U = \sum_{i=1}^{k} w_i F_i(x)$).

The simplicity (in implementation and use) and efficiency (from a computational perspective) are two of the most important advantages of this method that determined us to involve it in our approach.

The optimization algorithm advanced by our approach for solving TCP problem consists of the following steps:

(1) data synthesis;
(2) *active tests = filter(original tests)*;
(3) test prioritisation
   (a) identify a prioritisation of the *active tests*;
   (b) select, based on the order indicated by the obtained permutation, only the necessarily tests of the *active tests* that cover all the faults and all the requirements — *min tests = filter2(active tests)*.

In order to identify a prioritization of the *active tests* two types of algorithms have been considered: Greedy algorithm and an EA.

We have decided to involve the EAs in our approach due to their strengths: flexibility, robustness, and ability of solving various optimisation problems. Furthermore, the Greedy-based approach was reported as the current best solution single-objective optimisation method in TCP case in [10], [11], but latter was shown that EAs or the hybrid algorithms (a combination of EAs and Greedy) are able of identifying better prioritisations than the Greedy algorithms [21], [20], [11], [6].

In addition to the search algorithm, various optimisation criteria have been considered until now in the case of TCP: structural coverage [11], [8, 9, 15, 40], coverage of differences between two versions of program [10], time of last test execution [18], coverage augmented by human knowledge [38], and interaction coverage [4]. Some of the already proposed approaches are single objective [21], [5], [6], while others are multi-objective [20], [5], [6].

In this context, the third step of our approach consists of the optimal prioritization, being performed by using:

- a Greedy algorithm that sorts the *active tests* based on the cost associated to each test (Greedy-cost method).
- a GA that identifies an optimal permutation of the *active tests* following the next operations:
  - Generate a population of permutations and evaluate them based on:
    - The cost associated to each test (GA-cost method);
    - the minimal number of tests from the permutation that cover all faults and all the requirements (GA-noTests method);
    - more criteria: the number of useful tests, effectiveness, the efficiency (GA-TEE method).
  - Repeat the process of renewal population by genetic operators (selection, crossover, mutation);
  - choose the best permutation(s) from the last generation.

Remark. It worth mentioning that the Greedy Algorithm solution consists of a single permutation, whereas the GA Solution comprises the best permutation(s) from the last generation.

The novelty of our approach is related to the considered criteria: so far the fault detection capability was considered in the literature, but the effectiveness of a test case and the efficiency of a test case were not considered before. Furthermore, considering a multi-objective formulation with both perspectives was not investigated before.

In the next sections we will present the basic GA's ingredients: representation and fitness, while the search operators respect the standard details.

4.1 Chromosome Representation

A permutation-based representation is used for the chromosomes to encode a potential solution to TCP problem. The available tests (all tests from *active tests*) must be ordered, obtaining a prioritization of them. We will encode this order by using chromosomes with a number of genes equal to the number of tests, each gene being an element from a permutation.

For instance, we consider a simple example of TCP problem with a test suite T composed of $m = 10$ tests ($T = \{t_1, t_2, \ldots, t_{10}\}$), 8 requirements ($R = \{r_1, r_2, \ldots, r_8\}$) and 3 faults ($F = \{f_1, f_2, f_3\}$). The requirement matrix is given in Table 1 (the first 8 columns). The ninth column of Table 1 lists the normalised costs associated to all considered tests. The fault matrix is indicated by the last three columns in Table 1.

The modified requirements are r_2 and r_5. Dependent requirements are as follows: r_1 and r_8 depend on r_2; r_4, r_7, r_8 depend on r_5. In this context, $ReTests = \{t_4, t_6, t_9\}$ and $RegrTests = \{t_2, t_3, t_8, t_7, t_9$ (for r_2), t_5, t_7, t_8, t_9 (for r_5)$\}$. Therefore, *active tests* = $\{t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9\}$. A possible execution order of tests could be: $(t_7, t_2, t_6, t_8, t_9, t_5, t_4, t_3)$. We can observe that the first 5 tests from this permutation — t_7, t_2, t_6, t_8, t_9 — covered all changed/affected

Table 1: Requirement matrix and Fault matrix (example)

	r_1	r_2	r_3	r_4	r_5	r_6	r_7	r_8	cost	f_1	f_2	f_3
t_1	0	0	1	0	0	1	0	0	0.03	1	0	1
t_2	1	0	0	0	0	0	0	0	0.05	0	0	1
t_3	1	0	0	0	0	0	0	0	0.02	1	0	0
t_4	0	1	0	0	0	0	0	0	0.08	1	0	0
t_5	0	0	0	1	0	0	0	0	0.01	0	1	0
t_6	0	0	0	0	1	0	0	0	0.04	0	1	0
t_7	0	0	0	1	0	0	0	1	0.07	1	0	1
t_8	1	0	0	0	0	0	1	0	0.03	0	1	1
t_9	0	1	0	0	0	0	0	1	0.06	1	1	0
t_{10}	0	0	0	0	0	1	0	0	0.04	0	1	0

(modified and dependent) requirements. We denote these tests by *necessary tests*. The normalized cost associated to these *necessary tests* is 0.25 (while the cost associated to *active tests* is 0.36), while the total number of faults covered by them is 3.

4.2 Fitness

The quality of a potential order of tests is evaluated by taking into account: the correctness of the selection — all requirements are satisfied (modified requirements and dependent requirements) — and the efficiency of the selection — as few as possible tests, tests of minimal cost, tests of maximal fault, etc.

Therefore, the fitness function could take into account different metrics (some of them are frequently used in the testing domain, while others are borrowed from IoT domain). These metrics refer to:

- *the total cost of selected tests* (see Equation 5); because the (modified and dependent) requirements can be covered by few tests (*necessary tests*) than those from *active tests*, the total cost will be computed over these few tests only; in this case we deal with a minimization problem.

$$\text{Total\_Cost} = \sum_{t \in necessary\ tests} tc[t] \tag{5}$$

- *the number of tests that cover all the faults* (see Equation 6); in this case we deal with a minimisation problem

$$\text{Number\_Tests\_All\_Faults} = |\{TCAF\}| \tag{6}$$

  where $TCAF$ is the set of tests (from *active tests*) that cover all faults;
- *effectiveness for all test cases* (see Equation 2);
- *time based efficiency for all test cases* (see Equation 4).

As we already said, a possible solution for TCP described in Section 4.1 could be encoded as a permutation $(t_7, t_2, t_6, t_8, t_9, t_5, t_4, t_3)$. By taking into account the cost objective, we obtain: total cost = 0.25, number of covered faults = 3, number of required tests = 5, that means the actual solution is composed only by the first 5 tests $(t_7, t_2, t_6, t_8, t_9)$ as these five tests cover all the modified requirements (r_2 and r_5) and the dependent requirements (r_1, r_8 and r_4, r_7, r_8). In addition, these tests cover all faults. By taking into account the APFD objective we obtain a quality of 1.104. This time (and every time when APFD metric is considered as an objective function) all the tests are taken into account.

4.3 Genetic Operations

To automatically discover efficient sets of tests, biological-inspired operations (selection, crossover, and mutation) are applied over several generations respecting a steady-state approach [12].

Selection. Binary tournament selection is actually used in our approach. Two parents are randomly considered and the best of them, through the fitness function, is selected to be the first parent. The same procedure is repeated for selecting the second parent.

Crossover. The two selected chromosomes are recombined in crossover. For example, for two parents (two test permutations, (t_9,t_2,t_3,t_4,t_5,t_6,t_7,t_8) and (t_7,t_2,t_6,t_8,t_9,t_5,t_4,t_3) the new order of tests is obtained: (t_2,t_8,t_9,t_4,t_5,t_6,t_7,t_3).

Mutation. After crossover, the swap mutation operator is applied [12] when two elements of the permutation are interchanged. For example, a possible new permutation (t_9,t_2,t_3,t_4,t_5,t_6,t_7,t_8) is obtained from an old one (t_9,t_6,t_3,t_4,t_5,t_2,t_7,t_8) by mutation.

Section 5 addresses in more depth the different approaches in terms of required data, optimization criteria, achieved results, and outcome complementary analysis.

5 EXPERIMENTAL ANALYSIS

This section reports the experimental validation of the proposed TCP method, a steady-state Genetic Algorithm. In order to evaluate our approach, we have analyzed, in detail, four case studies.

5.1 Data for the Experimental Analysis

To evaluate our approach, we have analyzed, in detail, four case studies. Table 2 summarizes the specific properties of these cases: the number of requirements, the number of modified requirements, the number of dependent requirements, the number of tests and the number of faults per case study, respectively. More details about each case study are given in what follows.

Table 2: Specification of the considered case studies

Characteristics	Synthetic			Real
	CST	$CS_1 - replication$	CS_2	CS_3
# reqs	8	10	8	75
# depend reqs	4	1	3	10
# tests	10	30	50	470
# faults	3	10	15	18
# modif reqs	2	1	3	3

Based on [34], we have selected the case study method and conducted four case studies, the first three ones are theoretical ones and the last one is a real one. For each case study we specify the set of requirements, tests, modified requirements, a set of requirements dependencies related to the changes, number of faults and number of faults discovered.

5.1.1 Data for CST. The theoretical case study consists of eight requirements $R = \{r_1, r_2, \ldots, r_8\}$, ten test cases $T = \{t_1, t_2, \ldots, t_{10}\}$, three faults $F = \{f_1, f_2, f_3\}$ and one modified requirement $MR = \{r_2\}$. The requirements dependency matrix RDR is presented shortly by Table 1 (See Section 4.1). The other considered data elements of the case study are: the test traceability TR is depicted by Table 1 (See Section 4.1), the test cost C associated with each $t_j \in T, i = \overline{1, 10}$ is

presented as ninth column of the Table 1. The tests-faults matrix is depicted by the last three columns of Table 1 (See Section 4.1).

The data for this case study was presented in Section 4.1 to better explain and describe the elements for the EA approach.

5.1.2 Data for CS1-replication. Another theoretical case study consisting of 30 tests and 10 requirements is investigated. It represents an improved version of CS1 [32]. In the current paper, we use the data from CS1 [32] as a starting point to reason about the replication strategy discussed in Section 5.1.

5.1.3 Data for CS2. Another theoretical or synthetic case study composed from 50 tests and 10 requirements is investigated. It represents an improved version of CS1-replication. Three requirements were modified in the new version of the program (r_2, r_5, r_9). Three dependencies exist between requirements: r_3 and r_6 are influenced by r_2, r_{10} is influenced by r_9.

5.1.4 Data for CS3. Literature on RT techniques [36], [22] reports extended use of specific software repositories. The data of this case study is based on Software-artifact Infrastructure Repository (SIR), that consists of software used in different types of testing experiments [1].

Among the available software programs, named *objects* on SIR [1], **grep** object was selected. It met the prerequisite of having the test information in the required format and provides a high number of test cases in a test suite. For **grep** there is a peak of 470 test cases per test suite and 75 requirements. The changes from *version* v_0 to *version* v_1 consists of: i) 3 requirements were modified in the new version of the program: r_4, r_{58}, r_{68}; ii) 10 dependencies exist between requirements: r_3, r_{45}, r_{46}, r_{49}, r_{50}, r_{51}, r_{52} are influenced by r_{58}, and the requirements r_{60}, r_{62}, r_{64} are influenced by r_{68}.

To address the requirements dependencies of our case studies, several adjustments were needed on existing SIR tools.

Remark: The requirements dependencies are established at the start of the project and further in the design and implementation phase.

5.2 Settings of TCP Optimization

Several approaches were investigated for solving the TCP problem. Two of them use the Greedy algorithm for solving TCP problem, while the others are GA-based. The involved optimisation criteria are: the cost associated to a prioritisation; the number of tests that covered all the faults and changed requirements, and usability metrics (effectiveness, efficiency).

The optimisation methods are denoted as: Greedy-cost, Greedy-faults, GA-cost, GA-faults, GA-noTests, GA-APFD, GA-TEE (this is an optimisation that takes into account, in a weighted manner, more criteria: number of tests, effectiveness and efficiency). The GA parameters are set as follows: population size to 100, number of generations to 100, code length (number of possible tests) to a size of *ActualTests*, crossover probability to 0.8, and mutation probability to 0.1. Since the GAs involve randomness, more runs are performed in order to validate the obtained results.

5.3 Numerical Results

We conduct a first experiment to assess the performances of GA relative to the Greedy method. We are interested in evaluating to what extent the test case schedule obtained by GA is able to detect faults (effectiveness) earlier (lower execution cost) in comparison with Greedy-based technique. This reflects the developers' needs to discover regression faults with minimum cost. In the following, we present the conducted experiments that will help us answer to the research questions stated in Section 3.3.

5.3.1 Experiment for RQ1. GA versus Greedy. The results of our first study are presented in what follows. The answers to the research questions are also given. Table 3 reports the costs corresponding to the optimized permutation obtained by Greedy-cost and GA-cost on the investigated case studies. Furthermore, we present the number of tests that are required from the obtained permutation to cover all requirements and faults (the minimal criteria of correctness of a prioritization).

Table 3: Greedy *versus* GA (cost as optimisation criteria)

Case study	Greedy-cost		GA-cost	
	cost	no tests	cost	no tests
CST	0.21	6	0.14	4
CS1-replication	1.01	6	0.54	3
CS2	4.17	21	1.49	7
CS3	0.20	59	0.03	8

The analysis of Greedy-cost and GA-cost algorithm results shows that the later achieves usually lower costs than the former, indicating a better prioritization of the test cases. In addition, the test schedules identified by the GA are in general shorter than those identified by Greedy. For instance, in the study case CS2, with the Greedy algorithm, after sorting the tests by their cost, it constructs a permutation composed of 21 tests that cover all the changed requirements and faults, while the GA was able to identify a permutation composed by 7 tests, only).

5.3.2 Experiment for RQ2. GA-cost versus GA-noTests versus GA-TEE. The results of our second study are presented in what follows. The answers to the research questions are also given.

Table 4 reports the costs corresponding to the optimised permutation obtained by GA-cost, GA-noTests and GA-TEE, respectively, on the investigated case studies. Similar to the previous experiment, we present the number of tests that are required from the obtained permutation in order to cover all requirements and the faults.

Table 4: SO GAs *versus* MO GA (cost and number of required tests as optimisation criteria)

Study case	Single objective (SO)				Multi-objective (MO)	
	GA-cost		GA-noTests		GA-TEE	
	cost	no tests	cost	no tests	cost	no tests
CST	0.14	4	0.19	4	0.36	8
CS1-replication	0.54	3	0.58	3	2.01	8
CS2	1.49	7	1.68	7	3.97	16
CS3	0.03	7	0.03	8	0.04	12

By considering the cost associated with the identified optimal permutations, the experiments using the GA-cost approach reported the best results in terms of costs, followed by GA-noTests and GA-TEE. By considering the number of tests required to cover all requirements and all the faults, the experiments with GA-cost and GA-noTests reported close results, while GA-TEE identifies longer test schedules.

6 THREATS TO VALIDITY

Every experimental analysis may suffer from some threats to validity and biases that can affect the results of the study. In the following, we outline some issues which may have influenced the obtained results and their analysis are pointed out.

Threats to internal validity refer to the subjectivity introduced in setting the criteria for cost, number of tests and usability metrics in the multi-objective optimization algorithm. To minimize threats to internal validity, we conducted several combinations of experiments: GA versus Greedy considering cost, GA-noTests, GA-TEE respectively.

Threats to external validity are related to the generalization of the obtained results. Only four systems under test were considered for evaluation of the approach: three synthetic case studies and one open source project. We tried to reduce threats to external validity by considering various number of requirements, dependencies between requirements, number of tests, and number of faults.

7 CONCLUSIONS

In this paper, we investigated how different optimization techniques could improve the effectiveness of the testing activity, in particular the effectiveness of test case prioritization. We augmented the TCP definition by considering dependencies between requirements and also the test cases that are linked by requirements dependencies. We have applied both single and multi-criteria evolutionary approach, considering various perspectives (from structural code, fault detection to execution cost).

Future work will investigate TCS problem and will combine the results with the TCP approach. Thus, the obtained results are a prerequisite for our overall approach, the combination of two regression techniques to obtain the test suite according to the needs.

REFERENCES

[1] 2017. Software-artifact Infrastructure Repository. http://sir.unl.edu/. Accessed: 2016-06-06.

[2] ISO 9126. 2016. Software engineering – Product quality – Part 4: Quality in use metrics. https://www.iso.org/standard/39752.html. Accessed: 2015-11-1.

[3] Stephan Arlt, Tobias Morciniec, Andreas Podelski, and Silke Wagner. 2015. If A Fails, Can B Still Succeed? Inferring Dependencies between Test Results in Automotive System Testing. In *ICST*. 1–10.

[4] Renée C Bryce and Charles J Colbourn. 2006. Prioritized interaction testing for pair-wise coverage with seeding and constraints. *IST* 48, 10 (2006), 960–970.

[5] Dario Di Nucci, Annibale Panichella, Andy Zaidman, and Andrea De Lucia. 2015. Hypervolume-based search for test case prioritization. In *International Symposium on Search Based Software Engineering*. Springer, 157–172.

[6] Dario Di Nucci, Annibale Panichella, Andy Zaidman, and Andrea De Lucia. 2018. A Test Case Prioritization Genetic Algorithm guided by the Hypervolume Indicator. *IEEE TSE* (2018).

[7] Hyunsook Do, Siavash Mirarab, Ladan Tahvildari, and Gregg Rothermel. 2008. An Empirical Study of the Effect of Time Constraints on the Cost-benefits of Regression Testing. In *Proc. of FSE*. 71–82.

[8] Hyunsook Do, Gregg Rothermel, and Alex Kinneer. 2006. Prioritizing JUnit test cases: An empirical assessment and cost-benefits analysis. *Empirical Software Engineering* 11, 1 (2006), 33–70.

[9] S. Elbaum, Alexey G. Malishevsky, and G. Rothermel. 2000. Prioritizing Test Cases for Regression Testing. In *Proc. of ISSTA*. 102–112.

[10] Sebastian Elbaum, Alexey G Malishevsky, and Gregg Rothermel. 2002. Test case prioritization: A family of empirical studies. *IEEE TSE* 28, 2 (2002), 159–182.

[11] Michael G. Epitropakis, Shin Yoo, Mark Harman, and Edmund K. Burke. 2015. Empirical evaluation of pareto efficient multi-objective regression test case prioritisation. In *Proc. of ISSTA*, Michal Young and Tao Xie (Eds.). ACM, 234–245.

[12] D. E. Goldberg. 1989. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison Wesley.

[13] Mark Harman, Phil McMinn, Jerffeson Teixeira De Souza, and Shin Yoo. 2012. Search based software engineering: Techniques, taxonomy, tutorial. In *Empirical software engineering and verification*. Springer, 1–59.

[14] Md. Mahfuzul Islam, Alessandro Marchetto, Angelo Susi, and Giuseppe Scanniello. 2012. A Multi-Objective Technique to Prioritize Test Cases Based on Latent Semantic Indexing. In *2012 16th European Conference on Software Maintenance and Reengineering*. 21–30. https://doi.org/10.1109/CSMR.2012.13

[15] Dennis Jeffrey and Neelam Gupta. 2005. Test suite reduction with selective redundancy. In *Proc. of ICSM*. IEEE, 549–558.

[16] R. Kavitha and N. Sureshkumar. 2010. Test Case Prioritization for Regression Testing based on Severity of Fault. *International Journal on Computer Science and Engineering* 2, 5 (2010), 1462–1466.

[17] Alireza Khalilian, Mohammad Abdollahi Azgomi, and Yalda Fazlalizadeh. 2012. An Improved Method for Test Case Prioritization by Incorporating Historical Test Case Data. *Sci. Comput. Program.* 78, 1 (2012), 93–116.

[18] Jung-Min Kim and Adam Porter. 2002. A history-based test prioritization technique for regression testing in resource constrained environments. In *Proceedings of the International Conference on Software Engineering*. ACM, 119–129.

[19] Barbara Kitchenham. 2004. Procedures for Performing Systematic Reviews.

[20] Zheng Li, Yi Bian, Ruilian Zhao, and Jun Cheng. 2013. A fine-grained parallel multi-objective test case prioritization on gpu. In *International Symposium on Search Based Software Engineering*. Springer, 111–125.

[21] Zheng Li, Mark Harman, and Robert M. Hierons. 2007. Search Algorithms for Regression Test Case Prioritization. *IEEE TSE* 33, 4 (2007), 225–237.

[22] Chu-Ti Lin, Kai-Wei Tang, and Gregory M. Kapfhammer. 2014. Test suite reduction methods that decrease regression testing costs by identifying irreplaceable tests. *IST* 56, 10 (2014), 1322 – 1344.

[23] Alessandro Marchetto, Md. Mahfuzul Islam, Waseem Asghar, Angelo Susi, and Giuseppe Scanniello. 2016. A Multi-Objective Technique to Prioritize Test Cases. *IEEE TSE* 42, 10 (2016), 918–940. https://doi.org/10.1109/TSE.2015.2510633

[24] Mohammad Rava and Wan M.N. Wan-Kadir. 2016. A Review on Prioritization Techniques in Regression Testing. *International Journal of Software Engineering and Its Applications* 01, 1 (2016), 221–232.

[25] Gregg Rothermel, Mary Jean Harrold, Jeffery Ostrin, and Christie Hong. 1998. An empirical study of the effects of minimization on the fault detection capabilities of test suites. In *Proc. of ICSM*. 34–43.

[26] Gregg Rothermel, Roland H Untch, Chengyun Chu, and Mary Jean Harrold. 1999. Test case prioritization: an empirical study. In *Proc. of ICSM*. 179–188.

[27] Gregg Rothermel, Roland J. Untch, and Chengyun Chu. 2001. Prioritizing Test Cases For Regression Testing. *IEEE TSE* 27, 10 (2001), 929–948.

[28] Mazeiar Salehie, Sen Li, Ladan Tahvildari, Rozita Dara, Shimin Li, and Mark Moore. 2011. Prioritizing requirements-based regression test cases: A goal-driven practice. In *European Conference on Software Maintenance and Reengineering*. IEEE, 329–332.

[29] A. Singh. 2012. Prioritizing Test Cases in Regression testing using Fault Based Analysis. *International Journal of Computer Science Issues* 9, 2 (2012), 414–420.

[30] Hema Srikanth, Charitha Hettiarachchi, and Hyunsook Do. 2016. Requirements based test prioritization using risk factors: An industrial study. *IST* 69 (2016), 71 – 83.

[31] Fadel Touré, Mourad Badri, and Luc Lamontagne. 2014. A metrics suite for JUnit test code: a multiple case study on open source software. *J. Software Eng. R&D* 2 (2014), 14.

[32] Andreea Vescan, Camelia Serban, Camelia Chisalita-Cretu, and Laura Diosan. [n.d.]. On the use of Evolutionary Algorithms for Test Case Prioritization in Regression Testing considering Requirements Dependencies. = https://figshare.com/s/930f572f793829e07d40.

[33] Andreea Vescan, Camelia Şerban, Camelia Chisăliţă-Cretu, and Laura Dioşan. 2017. Requirement dependencies-based formal approach for test case prioritization in regression testing. In *2017 13th IEEE International Conference on Intelligent Computer Communication and Processing (ICCP)*. 181–188. https://doi.org/10.1109/ICCP.2017.8117002

[34] Robert K. Yin. 2009. *Case Study Research: Design and Methods*. SAGE Publications.

[35] Shin Yoo and Mark Harman. 2007. Pareto Efficient Multi-Objective Test Case Selection. In *Proc. of ISSTA*. ACM Press, London, United Kingdom, 140–150.

[36] Shin Yoo and Mark Harman. 2010. Using Hybrid Algorithm for Pareto Efficient Multi-objective Test Suite Minimisation. *J. Syst. Softw.* 83, 4 (2010), 689–701.

[37] Shin Yoo and Mark Harman. 2012. Regression testing minimization, selection and prioritization: a survey. *Softw. Test, Verif. Reliab* 22, 2 (2012), 67–120. http://dx.doi.org/10.1002/stv.430

[38] Shin Yoo, Mark Harman, Paolo Tonella, and Angelo Susi. 2009. Clustering test cases to achieve effective and scalable prioritisation incorporating expert knowledge. In *Proc. of ISSTA*. ACM, 201–212.

[39] Shin Yoo, Robert Nilsson, and Mark Harman. 2011. Faster Fault Finding at Google using Multi Objective Regression Test Optimisation. In *Proc .of ESEC/FSE*. 1–12.

[40] Lu Zhang, Shan-Shan Hou, Chao Guo, Tao Xie, and Hong Mei. 2009. Time-aware test-case prioritization using integer linear programming. In *Proc. of ISSTA*. ACM, 213–224.